

Lecture 6: Deployment, MLOps & Responsible AI

41-501
Applied AI

etienne@cmkl.ac.th

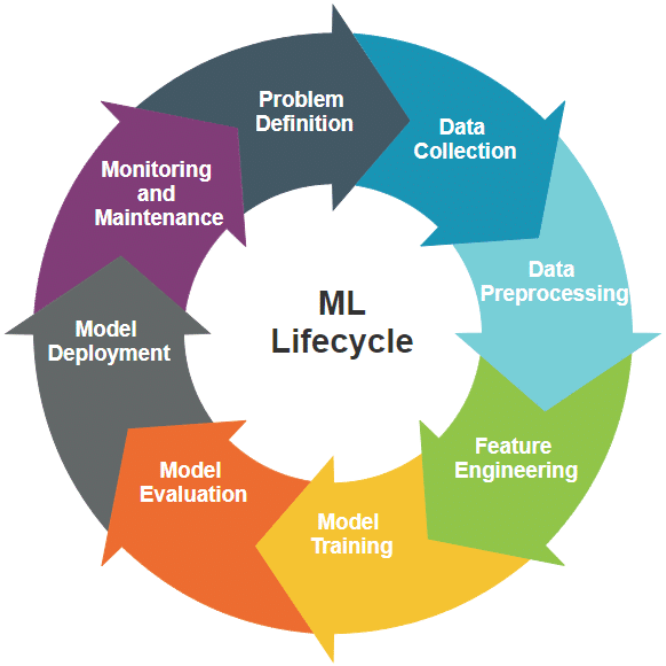
Content

- Deployment
- MLOps
- Responsible AI
- Lab



The Gap

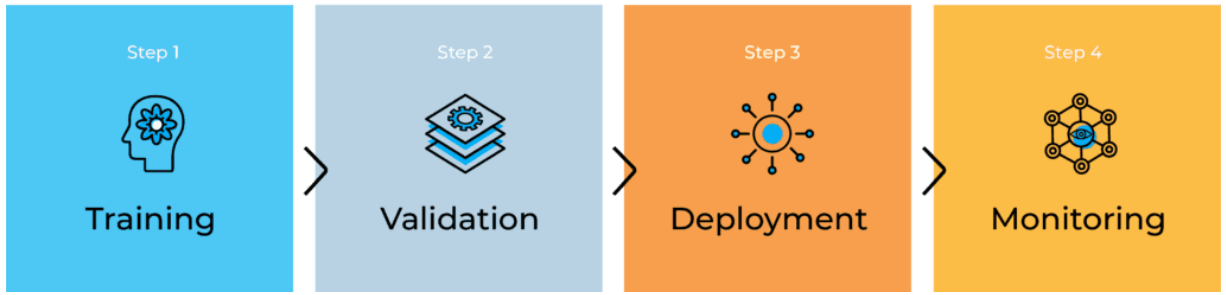
- ~60–90% ML projects never make it to production
- Common reasons
  - no clear business value
  - poor data quality
  - can’t reproduce results
  - no infrastructure to serve the model





What Does “Deployment” Mean?

- Making a trained model available to users or systems that need its predictions
- Could be:
  - a REST API
  - an embedded model in a mobile app
  - a batch job that scores overnight
  - a streaming pipeline
  - …



- From “I run the notebook” to “the system runs the model without me”



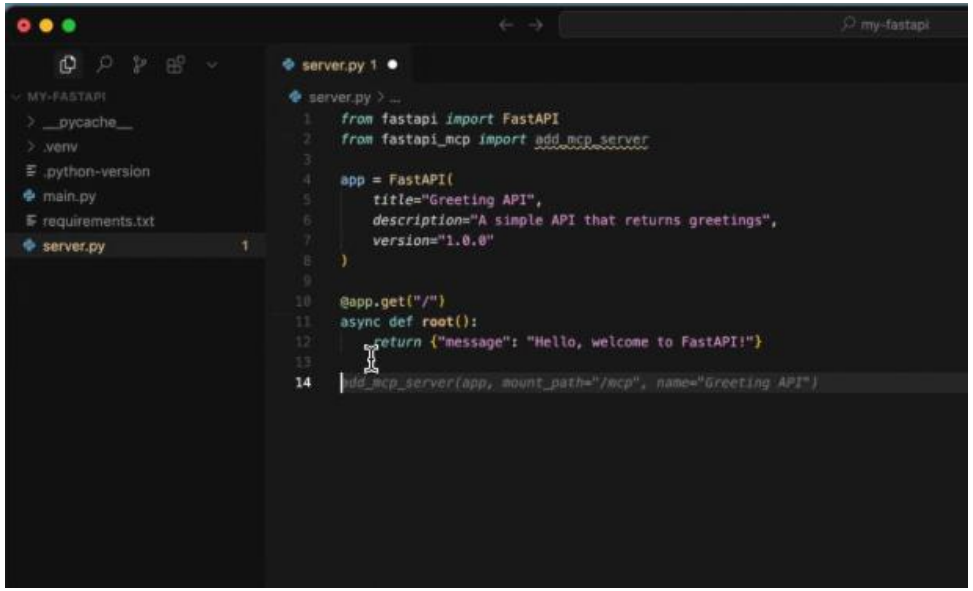
Model Serialization

- Trained models have to be saved in a portable format
- Common formats:
  - pickle (sklearn)
  - SavedModel (TensorFlow)
  - ONNX (cross-framework)
  - TorchScript (PyTorch)
  - joblib
- What gets saved:
  - model architecture + learned weights + preprocessing config



Model Serving: The Simplest Version

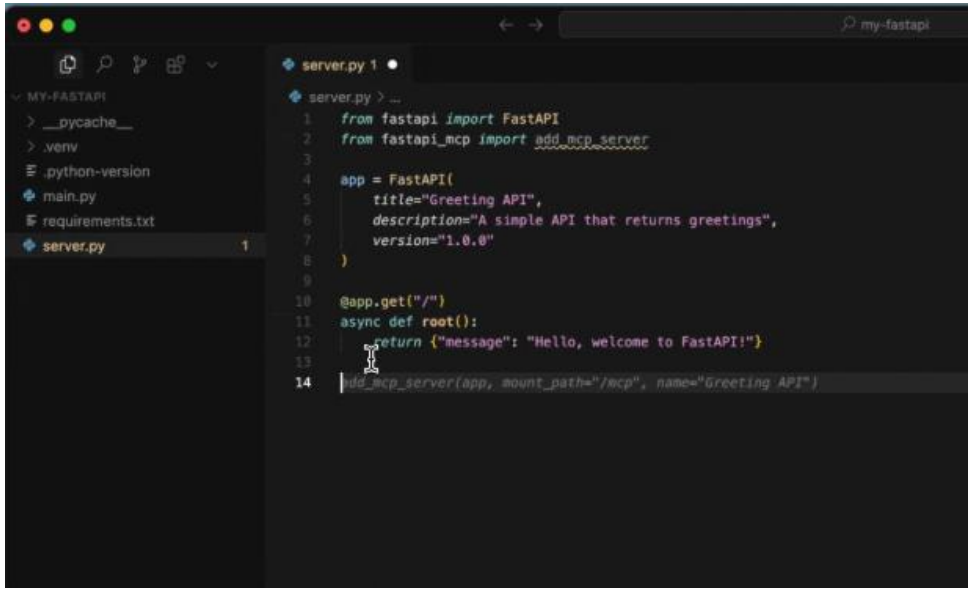
- Wrap your model's *.predict()* in a web endpoint
- Flask / FastAPI / BentoML: receive input as JSON, return prediction as JSON
- Example pseudocode:
  - POST /predict → receives {"features": [...]} → returns {"prediction": 0.87}
- Usually enough for internal tools and MVPs





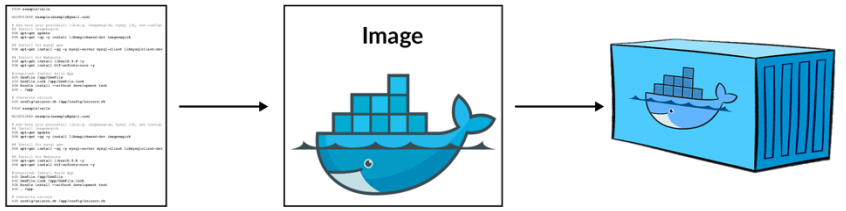
Model Serving: What Else?

- Authentication & rate limiting
- Input validation (what if someone sends garbage?)
- Latency requirements (500ms? 50ms?)
- Concurrent requests (is the model thread-safe?)
- Error handling (what happens when the model fails?)



Containerization with Docker

- Problem: “works on my machine”
  → different Python versions, missing libraries, …
- Solution: package everything into a container
  → code, model, dependencies, runtime



- Dockerfile basics: base image → install dependencies → copy model + code → expose port → run server
- Containers are reproducible, portable, and isolate your app from the host
- Deploy anywhere: **own server** vs. **managed options**:
  AWS SageMaker, Google Vertex AI, Azure ML, …
- Trade-off: more managed = less control, higher cost;
  less managed = more ops work



Batch vs. Real-Time Serving

- Real-time: user sends request, gets prediction back immediately (e.g., fraud detection at checkout)
- Batch: model scores a large dataset on a schedule (e.g., nightly churn predictions for all customers)
- Many production systems are actually batch → simpler, cheaper, easier to debug
- Does the user actually need a prediction in milliseconds?

Model Is Deployed… Now What?

- Models degrade over time; data drifts
- You need to monitor, retrain, and manage the lifecycle



Content

- Deployment
- MLOps
- Responsible AI
- Lab

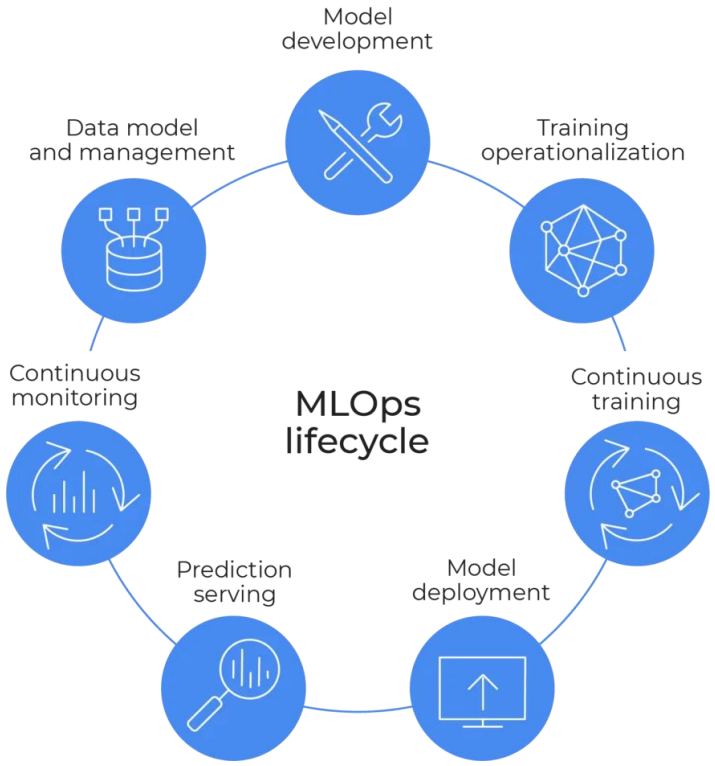


What Is MLOps?

- MLOps = Machine Learning + DevOps
- A set of practices for automating and managing the full ML lifecycle:

  data → training → deployment → monitoring → retraining

- Goals:
  - Reproducibility
  - Reliability
  - speed of iteration



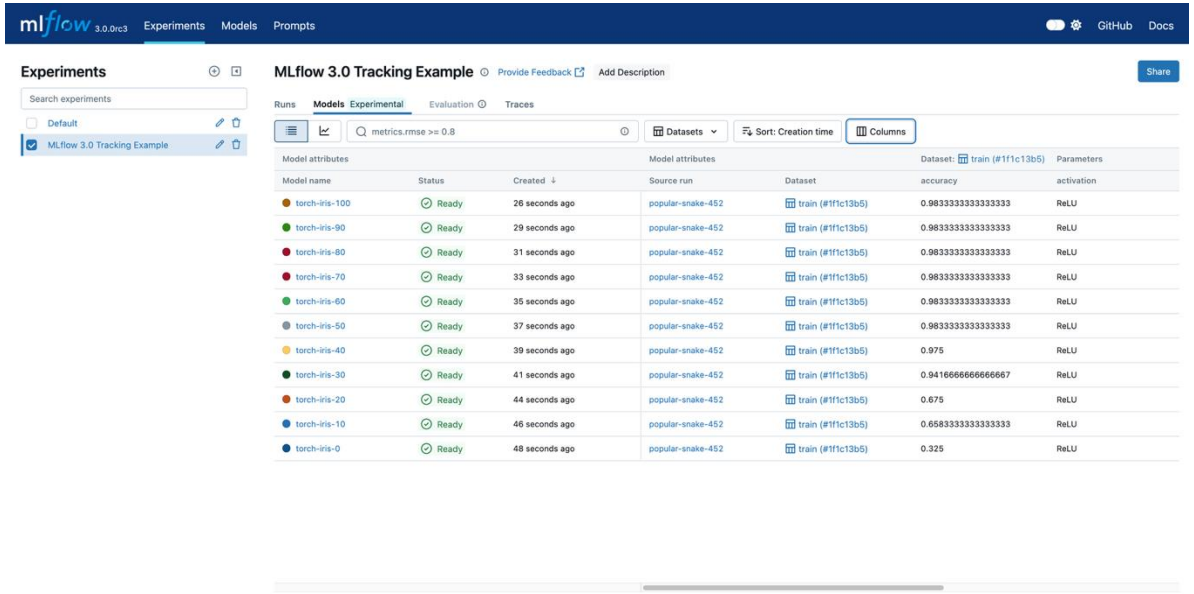
Reproducibility

- Can you reproduce last week's model exactly? If not, you have a problem.
- Three things to track:
  1. code (git)
  2. data (versioning)
  3. experiments (hyperparams + metrics)
- Tools:
  - Git for code
  - DVC or LakeFS for data versioning
  - MLflow or Weights & Biases for experiment tracking
- Minimum viable practice: log every training run with its config, data hash, and metrics



Experiment Tracking

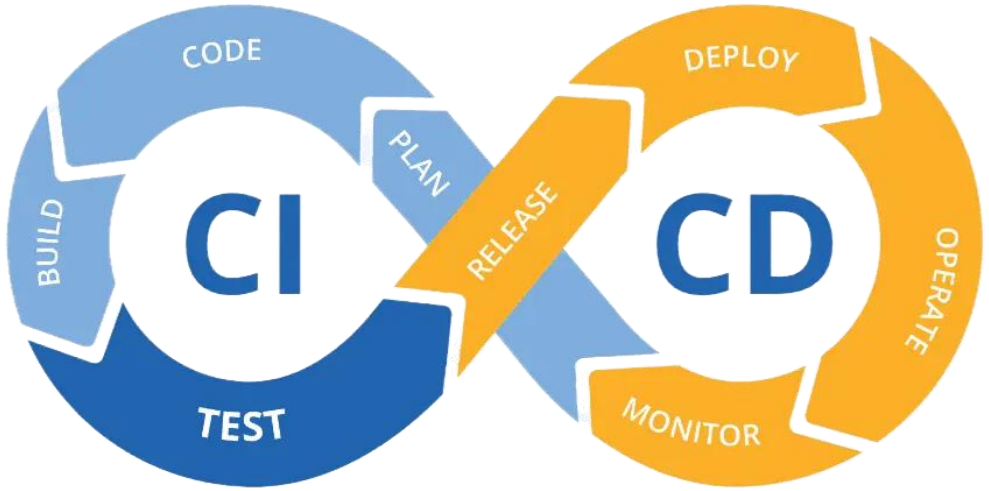
- Every time you train a model, log:
  - Hyperparameters
  - dataset version
  - metrics (accuracy, F1, AUC…)
  - model artifact
- MLflow example:
  ```
  mlflow.log_param("lr", 0.01),
  mlflow.log_metric("f1", 0.89),
  mlflow.sklearn.log_model(model)
  ```
- Creates a searchable history:
  - which combination worked best?
  - What changed between v3 and v4?
  - Etc.





CI/CD for ML

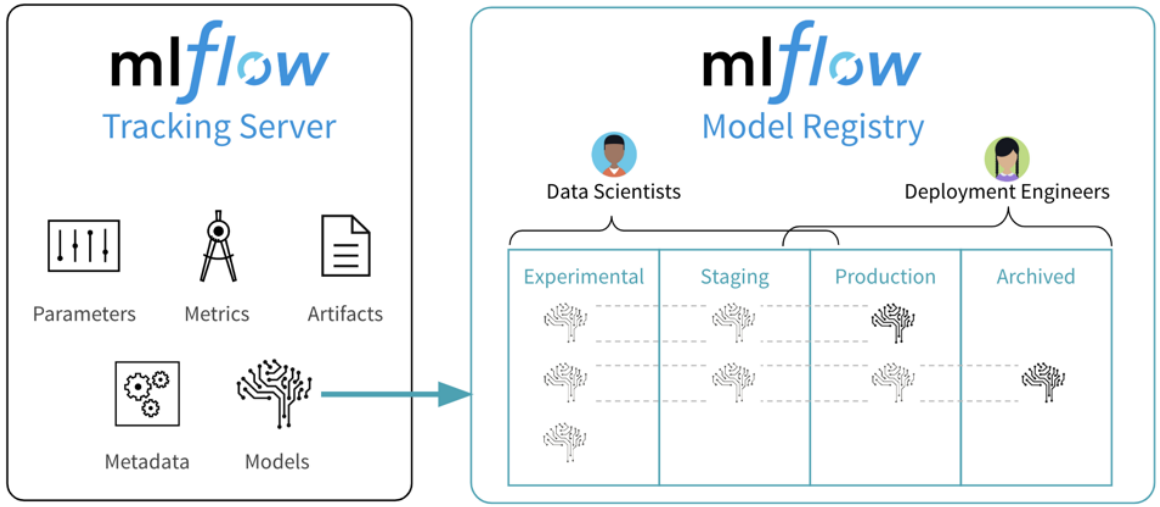
- Continuous Integration/Continuous Deployment
- Software engineering: code change → automated tests → deploy
- ML: code/data change → automated tests + retraining → validate → deploy
- ML-specific tests:
  - data schema validation
  - model performance thresholds
  - prediction distribution checks
- Prevent bad models from reaching production automatically





Model Registry

- A central place to store, version, and manage trained models
- Each model has a version, metadata (who trained it, on what data, what metrics), and a stage (staging / production / archived)
- Promotes a model from “staging” to “production” only after validation
- Tools:
  - MLflow Model Registry
  - SageMaker Model Registry
  - Vertex AI Model Registry



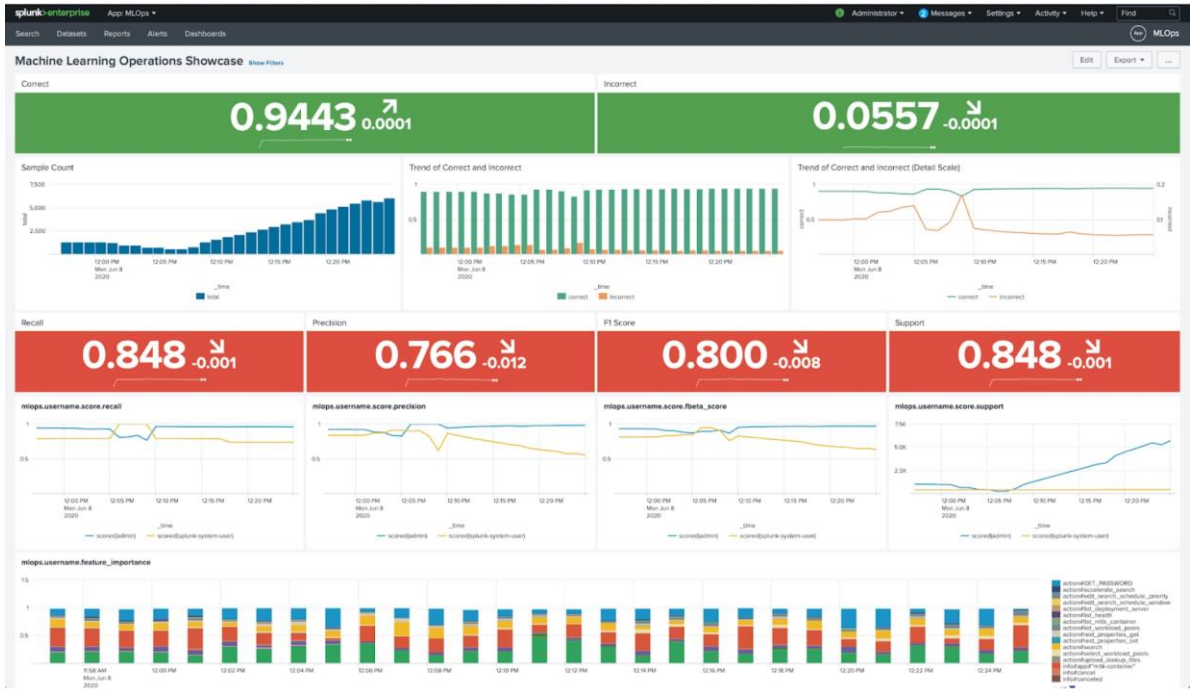
Monitoring

- Data changes:
  - customer behavior shifts
  - new products launch
  - seasons change
  - …
- Data drift: the input distribution shifts away from training data
- Concept drift: the relationship between inputs and outputs changes
- Example: a fraud model trained pre-COVID fails during COVID because spending patterns changed dramatically



What to Monitor?

- **Model performance metrics**
  if you have ground truth labels → often delayed
- **Prediction distribution**:
  are outputs shifting? (e.g., suddenly predicting 90% positive)
- **Input feature distributions**: statistical tests (KS test, PSI) to detect drift
- **System metrics**:
  latency, throughput, error rates, memory usage
- Set up **alerts** for anomalies on each



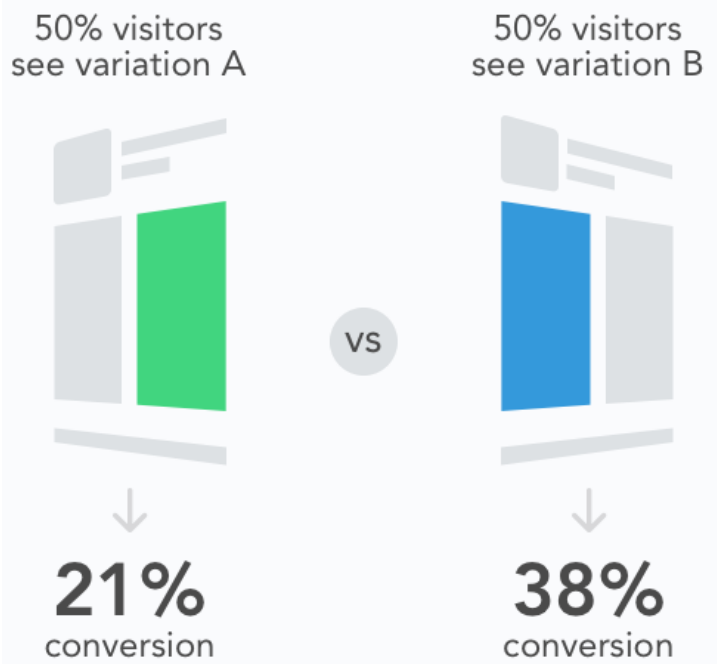
When to Retrain?

- Scheduled: every week/month, regardless (simple but wasteful)
- Triggered: when monitoring detects drift or performance drop
- Continuous: new data flows in, model retrains automatically (advanced)
- Key principle: always compare the new model against the current production model before swapping



A/B Testing for Models

- Don’t just swap models → test them against each other
- Route a percentage of traffic to the new model, compare real-world metrics
- Same principles as Week 5
- Shadow mode: new model runs in parallel but its predictions aren’t shown to users yet → lets you compare without risk





MLOps Maturity

- Level 0: Manual → notebook, manual deployment, no monitoring
- Level 1: ML pipeline automation → automated training, experiment tracking, basic monitoring
- Level 2: CI/CD pipeline automation → automated testing, model validation, automated deployment
- Most teams are at Level 0 or 1; Level 2 is the goal

level			level	
Microsoft	0	No MLOps	0	Google
	1	DevOps, no MLOps		
	2	Automated training	1	
	3	Automated model deployment		
	4	Full MLOps Automated Operations	2	



Content

- Deployment
- MLOps
- Responsible AI
- Lab



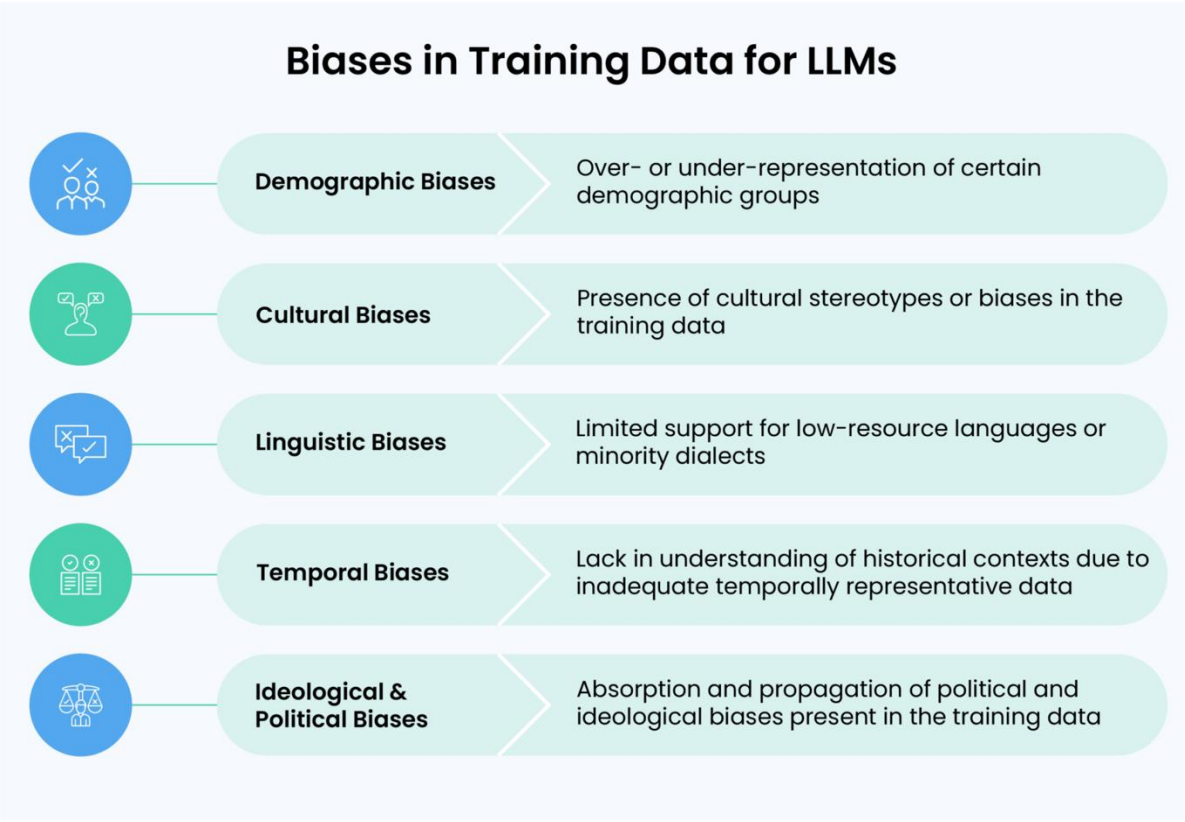
Responsible AI

- AI systems make decisions that affect people: hiring, lending, healthcare, criminal justice
- “Responsible AI” is not a separate activity → it’s part of every step we covered today
- Three lenses:
  - fairness (is it biased?)
  - transparency (can we explain it?)
  - governance (who is accountable?)
- Not just ethics → it’s engineering practice, legal compliance, and risk management



Where Bias Enters the Pipeline

- Training data: historical data reflects historical inequities (e.g., hiring data from a biased process)
- Feature selection: using proxies for protected attributes (zip code → race, name → gender)
- Labeling: human annotators carry their own biases
- Evaluation: optimizing for overall accuracy can hide poor performance on minority groups
- Deployment: feedback loops can amplify existing biases



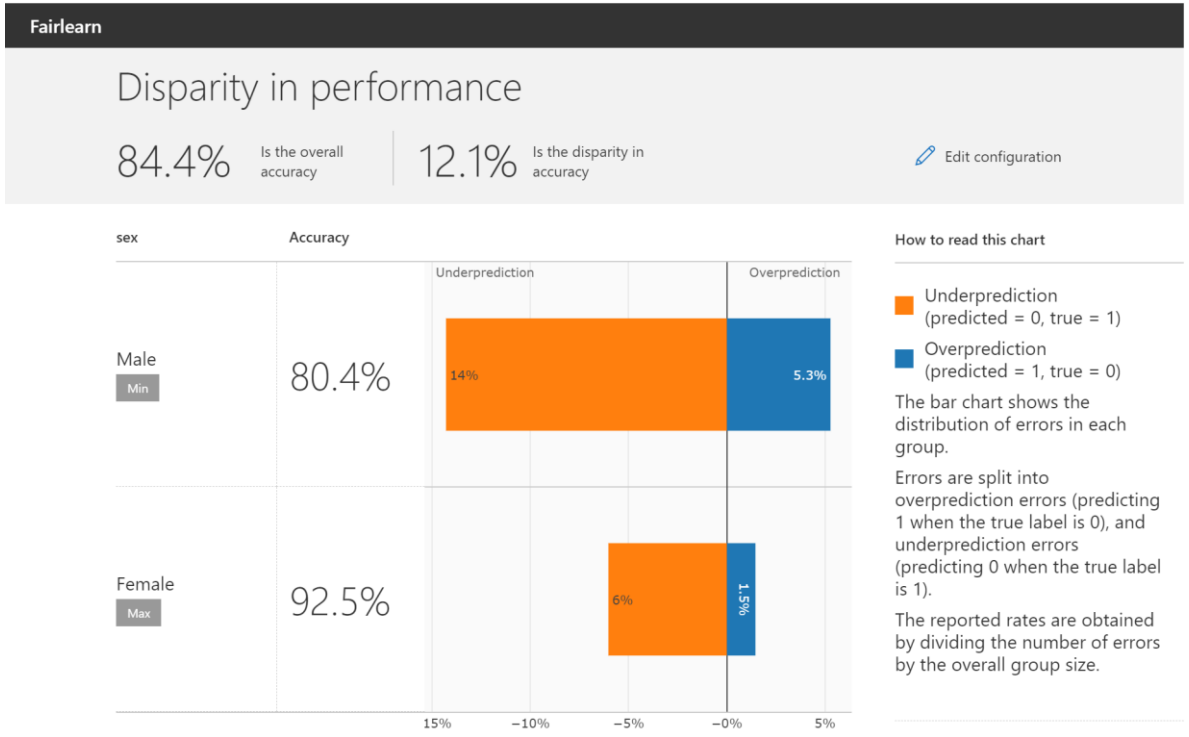
Fairness Metrics

- Demographic parity: prediction rates are equal across groups
- Equalized odds: true positive and false positive rates are equal across groups
- Individual fairness: similar individuals receive similar predictions
- Problem: these definitions can contradict each other → you cannot satisfy all of them simultaneously (impossibility theorems)
- The right metric depends on the context and what kind of harm you're trying to prevent



Fairness Metric Tools

- **Fairlearn** (Microsoft): bias metrics + mitigation algorithms for sklearn-compatible models
- **AI Fairness 360** (IBM): comprehensive toolkit with 70+ metrics
- **What-If Tool** (Google): interactive visual exploration of model fairness
- Workflow: define protected attributes → compute group-level metrics → compare across groups → if gaps exist, decide on mitigation



Bias Mitigation Strategies

- Pre-processing: rebalance or re-weight training data
- In-processing: add fairness constraints during training (e.g., adversarial debiasing)
- Post-processing: adjust prediction thresholds per group to equalize a fairness metric
- Trade-offs: mitigation often reduces overall accuracy slightly → this is an explicit trade-off to discuss with stakeholders, not a purely technical decision



Model Cards & Documentation

- Model cards (Mitchell et al., 2018): standardized documentation for trained models
- Contents: intended use, training data summary, performance across demographic groups, limitations, ethical considerations
- “Nutrition label” for ML models
- Becoming industry standard: Hugging Face requires model cards for all hosted models

Model Card

- **Model Details**. Basic information about the model.
  - Person or organization developing model
  - Model date
  - Model version
  - Model type
  - Information about training algorithms, parameters, fairness constraints or other applied approaches, and features
  - Paper or other resource for more information
  - Citation details
  - License
  - Where to send questions or comments about the model
- **Intended Use**. Use cases that were envisioned during development.
  - Primary intended uses
  - Primary intended users
  - Out-of-scope use cases
- **Factors**. Factors could include demographic or phenotypic groups, environmental conditions, technical attributes, or others listed in Section 4.3.
  - Relevant factors
  - Evaluation factors
- **Metrics**. Metrics should be chosen to reflect potential real-world impacts of the model.
  - Model performance measures
  - Decision thresholds
  - Variation approaches
- **Evaluation Data**. Details on the dataset(s) used for the quantitative analyses in the card.
  - Datasets
  - Motivation
  - Preprocessing
- **Training Data**. May not be possible to provide in practice. When possible, this section should mirror Evaluation Data. If such detail is not possible, minimal allowable information should be provided here, such as details of the distribution over various factors in the training datasets.
- **Quantitative Analyses**
  - Unitary results
  - Intersectional results
- **Ethical Considerations**
- **Caveats and Recommendations**

The Regulatory Landscape

- EU AI Act (2024): risk-based classification
  - unacceptable risk (banned)
  - high risk (strict requirements)
  - limited risk (transparency obligations)
  - minimal risk (no regulation)
- High-risk examples: hiring tools, credit scoring, medical devices, law enforcement
- Requirements for high-risk: data governance, documentation, human oversight, robustness testing
- Other frameworks: NIST AI RMF (US), Singapore's Model AI Governance Framework
- (Changes all the time)



Responsible AI as Engineering Practice

Should be integrated into the MLOps workflow:

- Data collection: audit for representation, document sources
- Training: track fairness metrics alongside accuracy
- Deployment: monitor for drift in fairness metrics, not just accuracy
- Organization: clear ownership, incident response plans, regular audits
- Your responsibility as an AI practitioner: if you build it, you are accountable for how it behaves



Content

- Deployment
- MLOps
- Responsible AI
- Lab



Lab

Think about your final project:

- How should it be deployed?
- Tracking of Experiments?
- How would CI/CD look like?
- What should be monitored?
- When/how would you retrain?
- What level of MLOps Maturity are you at?
- Any bias in your data?
- How would you ensure fairness in your model after deployment?
- …



ขอบคุณครับ

Thank you!

etienne@cmkl.ac.th

